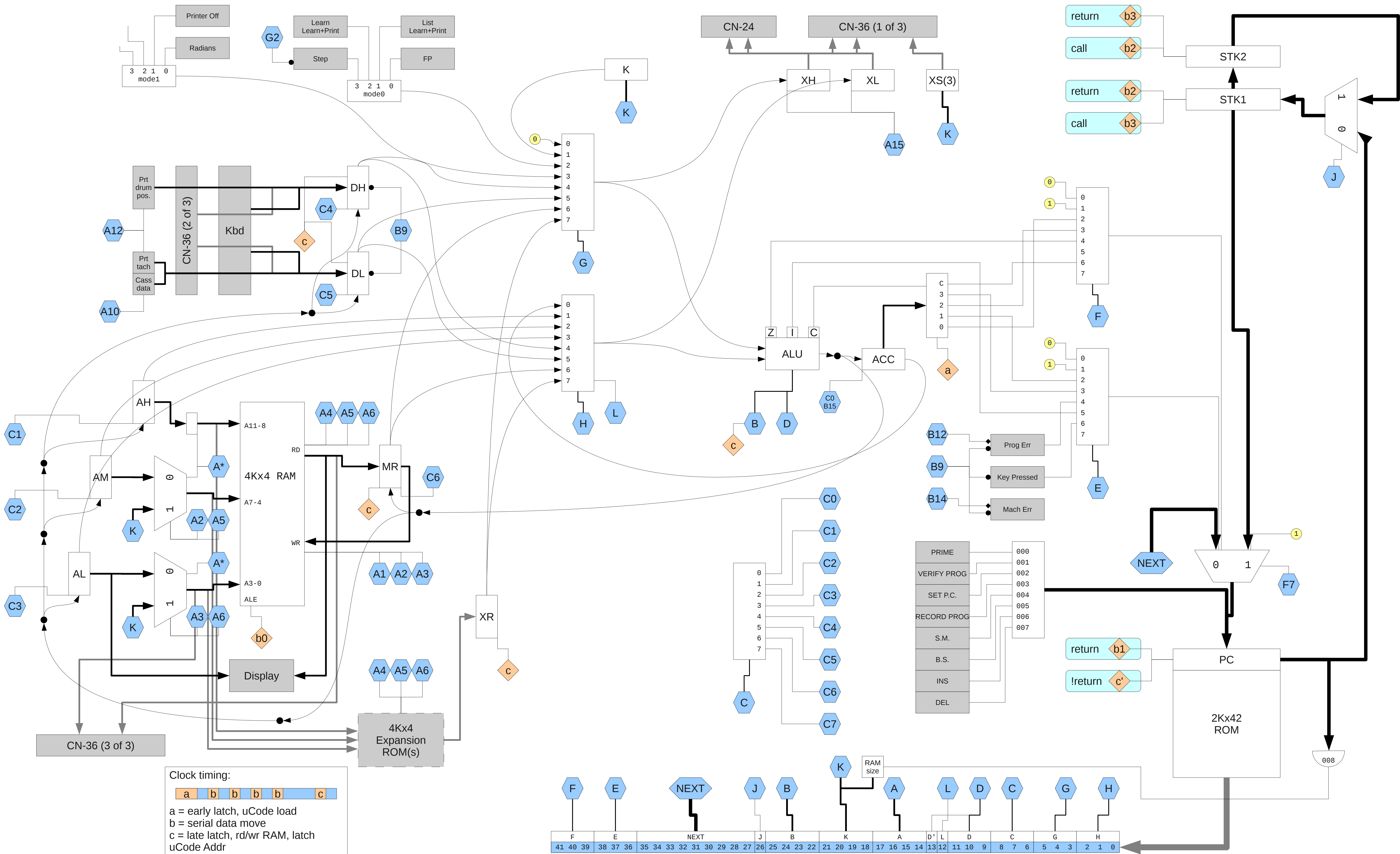




A op:
0: NOP?
1: wr(AH,AM,AL)
2: wr(15,15,AL)
3: wr(15,15, K)
4: rd(AH,AM,AL)
5: rd(15,15,AL)
6: rd(15,15, K)
7: Ld Prtr Hammer bits
8: Printer Feed
9: ?
10: DL = periph
11: Tape Data = bit 0 (DL)
12: DH,DL = Prt/Tape Sts
13: Tape Motor ON
14: Tape Motor OFF
15: XH,XL = <G>,<H>

B op:
0: NOP?
1: set bit 0 (ACC)
2: set bit 1 (ACC)
3: set bit 2 (ACC)
4: set bit 3 (ACC)
5: res bit 0 (ACC)
6: res bit 1 (ACC)
7: res bit 2 (ACC)
8: res bit 3 (ACC)
9: clear 6184
10: bit 0 (ACC) = IZ
11: bit 1 (ACC) = Z
12: Set Prog Err
13: clear ACC
14: Set Mach Err
15: ACC = (ALU) [if C0]

C op:
0: ACC = (ALU) [if B15]
1: AH = (ALU)
2: AM = (ALU)
3: AL = (ALU)
4: DH = (ALU)
5: DL = (ALU)
6: MR = (ALU)
7: NOP?

D op: (D'==0)
0: ALU = <G> + <H>
1: ALU = <G> + <H> + 1
2: ALU = <G> + <H>
3: ALU = <G> + <H> + C
4: ALU = <G> + <H> + 1
5: ALU = <G> & <H>
6: ALU = <G> | <H>
7: ALU = 0b0000

D op: (D'==1)
0: ALU = <H> - <G>
1: ALU = <H> - <G> - 1
2: ALU = <H> - <G>
3: ALU = <H> - <G> - C
4: ALU = <H> - <G> - 1
5: ALU = <G> & <H>
6: ALU = <G> ^ <H>
7: ALU = 0b0000

*0-6 update Z flag
0-4 update I flag
2-4 update C flag*

E op: conditional branch bit1
0: bit 0 (NEXT) = 0
1: bit 0 (NEXT) = 1
2: bit 0 (NEXT) = bit 0 (ACC)
3: bit 0 (NEXT) = bit 2 (ACC)
4: bit 0 (NEXT) = Prog Err
5: bit 0 (NEXT) = I
6: bit 0 (NEXT) = Any Key Down
7: bit 0 (NEXT) = Return

F op: conditional branch bit0
0: bit 1 (NEXT) = 0
1: bit 1 (NEXT) = 1
2: bit 1 (NEXT) = bit 1 (ACC)
3: bit 1 (NEXT) = bit 3 (ACC)
4: bit 1 (NEXT) = Z
5: bit 1 (NEXT) = ?
6: bit 1 (NEXT) = C
7: bit 1 (NEXT) = ?

G op:
0: 0b0000
1: K
2: mode 0*
3: mode 1
4: DH
5: DL
6: MR
7: XR

* Clears STEP

H op:
0: ACC
1: AH
2: AM
3: AL
4: DH
5: DL
6: MR
7: XR

L op 0: 0b0000

J op: (F!=7)
0: jump
1: call

K op:
K = <imm>*

RAM size if
PC=008

L op:
0: <H> = 0b0000
1: <H> = <H>

Keyboard direct jump
(microcode address)

PRIME	000
Verify Prog	001
Set PC	002
Rec Prog	003
Search Mk	004
B.S.	005
Insert	006
Delete	007

RAM Addresses	Reg #	Prog Steps
0xffff - 0xff0	15 15	
0xfef - 0xfe0	15 14	
0xfd - 0xfd0	15 13	
0xfc - 0xfc0	15 12	
0xfb - 0xfb0	15 11	
0xfa - 0xfa0	15 10	
0xf9 - 0xf90	15 09	
0xf8 - 0xf80	15 08	
0xf7 - 0xf70	15 07	
0xf6 - 0xf60	15 06	0000-0007
0xf5 - 0xf50	15 05	0008-0015
0xf4 - 0xf40	15 04	0016-0023
0xf3 - 0xf30	15 03	0024-0031

...	...	...
0x13 - 0x130	01 03	1816-1823
0x12 - 0x120	01 02	1824-1831
0x11 - 0x110	01 01	1832-1839
0x10 - 0x100	01 00	1840-1847
0x0ff - 0x0f0	00 15	
0x0ef - 0x0e0	00 14	
0x0df - 0x0d0	00 13	
0x0cf - 0x0c0	00 12	
0x0bf - 0x0b0	00 11	
0x0af - 0x0a0	00 10	
0x09 - 0x090	00 09	
0x08 - 0x080	00 08	
0x07 - 0x070	00 07	
0x06 - 0x060	00 06	
0x05 - 0x050	00 05	
0x04 - 0x040	00 04	
0x03 - 0x030	00 03	
0x02 - 0x020	00 02	
0x01 - 0x010	00 01	
0x00 - 0x000	00 00	

System register usage

15 15 System memory
15 14 ?
15 13 "Scientific" display
15 12 "Floating Point" display
15 11 "Learn Mode" display
15 10 ?
15 09 (used for pi)
15 08 Prog Stack (4 words/entry)
15 07 Prog Stack

Prog Stack starts with 00 of 15 08 and proceeds to 15 of 15 08, then continues at 00 of 15 07.

Note, the system does not prevent use of these "registers", but using them almost certainly causes Mach Errs.

System memory "15 15"

15 bit0=Keytrace
14 (2..11 for prog step lo)
13 bit0=ProgTrace, bit1=GroupI/O
12 ?
11 Prog Stack Depth
10 ?
09 ?
08 Keyboard code, lo
07 Keyboard code, hi
06 ?
05 ?
04 ?
03 bit0=Run, bit1=GroupI/O
bit2=ROM
02 Prog Step (RAM adr lo)
01 Prog Step (RAM adr mi)
00 Prog Step (RAM adr hi)

Display segment decoder

(except digits 0 and 13)

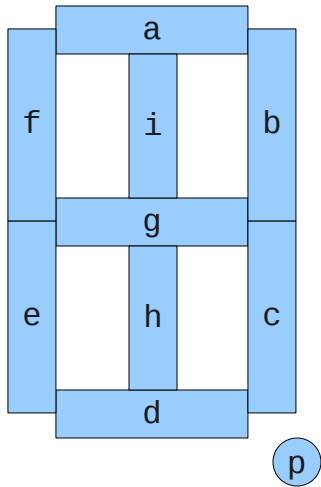
val	segments	symbol
00	a b c d e f - - -	'0'
01	- - - - - h i -	'1'
02	a b - d e - g - -	'2'
03	a b c d - - g - -	'3'
04	- b c - - f g - -	'4'
05	a - c d - f g - -	'5'
06	- - c d e f g - -	'6'
07	a b c - - - - -	'7'
08	a b c d e f g - -	'8'
09	a b c - - f g - -	'9'
10	- - - - - - - p	'.'
11	- - c d - g - -	'>'
12	- b - - - f g - -	'u'
13	a - - d - f g - -	'<'
14	- - - d e f g - -	'E'
15	- - - - - - - -	' '

Digits 0 and 13:

11	- - - - - g - -	'.'
12	- - - - - g h i -	'+'

Digit positions map directly to AL of register "15 11"

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15



Printer drum column-symbol decoder

val	0-15	16	17	18	19	20
00	0	E	0	S	M	X
01	1	T	1	RE	ST	Y
02	2	+	2	W	α	Z
03	3	-	3	Go	SP	A
04	4	×	4	J _o	J _θ	B
05	5	÷	5	J ₊	J _e	C
06	6	ST	6	SN	S ⁻¹	D
07	7	RE	7	CS	C ⁻¹	E
08	8	*	8	TN	T ⁻¹	F
09	9	*	9	RD	DR	G
10	.	f	10	LN	LG	H
11	o	F	11	e ^x	10 ^x	I
12	***	A	12	x ²	I	J
13	+	B	13	√x	x	K
14	-	C	14	LP	EP	L
15	Δ	D	15	¹ /x	RT	M

*** spells "...OVERFLOW..." across the 16 columns